

Lamina

Source-aware production methods for document workflows

Lamina Project

21 September 2026 — technical design and implementation status

Abstract

Teams increasingly ask language models to turn large, changing source collections into guides, briefs, references, questions, and other deliverables. A model can read and write, but an operational workflow must still decide which sources have authority, what information must survive compression, which workers require shared context, what can run concurrently, and what must be repaired after a source or review finding changes. Lamina is open-source infrastructure for these decisions. It now includes a source-linked guide builder, a separate reusable method runtime, and a source-aware production engine connected to a local project interface, CLI, exports, and persisted recovery. This paper defines a broader task-shaped architecture without assuming that every output is a lesson or that a dependency graph alone is the method. It gives operational definitions for source ownership, context selection, dependency and resource edges, cache identity, and bounded repair. A local synthetic benchmark demonstrates scheduler and cache behavior only; semantic quality, production latency, and competitive advantage require a new-source, quality-matched evaluation. The main product hypothesis is that an inspectable source-to-decision-to-artifact trail can reduce the work of producing and safely updating useful documents. That hypothesis is not yet established by the present fixtures.

1 The production problem

A team preparing an operating brief may have a current procedure, a superseded note, a table of exceptions, and a prior document used only as a style example. It needs a result that can be used, not a plausible summary. A single model call can blend the old and new rules, lose a condition in a table, or write a clean draft whose source basis is hard to revise. Sending the whole collection to every writer increases input cost and still does not ensure that the critical condition is attended to; long-context retrieval is known to vary with position and task [7]. The failure may originate before writing, when a parser, reader, or planner narrows what later workers are allowed to see.

The customer value is clearest after change. If a rule or document is revised, the owner must know what the earlier artifact depended on, which decisions are invalid, what can be reused, and whether the replacement reached the intended surface. Recreating the entire document is expensive and can introduce unrelated errors. Editing the final sentence alone can leave a false upstream interpretation available for the next release. Lamina’s intended unit of work is therefore a *production method*: an explicit, inspectable way to transform a defined source set into a defined artifact, with decisions and repair paths retained.

This is neither a claim that graphs or parallel agents are new nor a claim that bookkeeping can replace editorial judgment. Parallel function execution has been studied in LLM systems [5]; graph-based

agent workflows, durable state, and interruptions are available in established tools [2, 6]. The distinct question here is whether a source-aware production contract and a record of semantic decisions make document change and repair less costly at matched output quality. That question requires testing.

2 Evidence from several output forms

Lamina’s design was informed by document and learning-product work with incompatible output contracts. A differential reference must own each retrieval target, preserve meaningful variants, and make the answer to a presenting problem easy to find. A spoken lesson must preserve lists, explanation, useful repetition, and actual playable delivery. An interactive case must keep candidate-visible findings separate from an examiner’s answers and reveal new information after actions or elapsed time. A written case generator adds a separate setter, question-specific answer keys, and a blind solver that sees only the staged information available to a candidate at each turn. A technical brief needs a defensible argument and a record of conflicting sources. They can share infrastructure for sources, work allocation, provenance, and revision, but they should not share one mandatory narrative template.

Three distinctions recur across these jobs. First, *source consideration* is not final coverage: a model may read an item and correctly omit it, or may never see an item that it later claims to have considered. Second, a source quotation is evidence of provenance, not proof that the sentence citing it is entailed, correct, or useful. Third, an artifact’s schema and its user experience are separate: a valid script is not audited audio; a well-formed case is not necessarily a realistic encounter; a PDF’s page count does not establish that its diagrams are legible. These distinctions determine what receipts and tests are needed at each boundary.

3 A task contract before a graph

A job begins with a task contract

$$\mathcal{T} = (G, A, S, P, O, Q, R), \tag{1}$$

where G is the goal, A the audience or operator, S the accepted source set, P the policy governing each source’s use, O the target artifact and its delivery surface, Q the acceptance criteria, and R the time, model, money, context, and machine-resource limits. This is an engineering interface, not a formula that scores document quality. If any component is missing, a method may still run, but its success claim is correspondingly weaker.

Source policy is more precise than a generic ‘included’ flag. A current specification may establish an operational rule; an old note may document a conflict but lack present authority; a style exemplar may influence tone but not factual claims; held-out questions must remain out of generation prompts. A source has an identity, version or hash, role, and locator. The policy records what the system is permitted to infer from it. A later change in policy is a semantic input change even when the underlying file bytes are unchanged.

An assessment question can legitimately serve as an example of *question form* while being prohibited as factual teaching evidence. Another question can be protected as a held-out evaluation item. These roles cannot be inferred from a filename. A manifest should distinguish sources discovered, inventoried,

actually read, selected as factual evidence, used as form exemplars, and withheld. A source count at one boundary must not be reported as comprehensive use at another.

The intended representation follows the task. For a guide it may be claims, conditions, exceptions, causal steps, and contrasts. For a differential reference it may be presentation–answer objects with one owner per retrieval target and distinct variants when the question genuinely changes. For an interactive case it may be states, candidate requests, released findings, actions, consequences, and marking opportunities. Figure and table regions require modality-aware extraction and verification. The general rule is to choose a representation that preserves the distinctions needed to judge the final artifact. A universal ‘concept’ type is convenient but can hide the answer unit or behavior that actually matters.

This also changes review. A document claim may be reviewed for support and contradiction. A differential object needs its answer family, exclusions, qualifiers, and retrieval target checked. An interactive case requires a state-machine review: can the candidate obtain an indicated finding, does a treatment alter the next observation, and does the examiner possess the data required to answer reasonable questions? A single generic ‘pass’ score collapses these into an unhelpful average. The method should declare the domain of each check and whether failure blocks delivery, creates a review flag, or opens a source-policy decision. A model reviewer can help find defects; its response is not an independent authority over a clinical, legal, or financial claim.

A model can propose objects and relations, while code owns stable IDs, source locators, exact membership, and declared limits. Relations such as *same target*, *variant*, *contrast*, *prerequisite*, and *conflict* should be typed rather than collapsed into a single semantic merge. An ambiguous merge should remain unresolved for review. A selected object can be written, deferred, deliberately omitted, or left open; no universal rule should force all source items into prose. The acceptance criteria decide how much omission accounting is justified by the task.

Two limits prevent this representation from becoming a decorative ontology. An object must have a downstream consumer: a writer needs it to explain, a reviewer needs it to test, or a repair operation needs it to locate a decision. If no consumer uses an added field, it should not be required. Likewise, exact structural accounting should be paired with a semantic counterexample. A validator can prove that every extracted item has an owner while a merged item silently loses an exception. A source-bound object ledger should therefore expose the original region and the model’s claimed transformation, allowing an independent reviewer to sample risky merges and central answers rather than trusting item counts.

4 Source windows and ownership

Reading a large source requires bounded packets, but a boundary can split a definition from its condition. Let the accepted regions be $S_1, \dots, S_n$. A reading job j receives a focal core C_j and neighboring context H_j . The reader may use the halo to interpret the core but owns extraction only from C_j . For an exact partition of readable core regions,

$$\bigcup_j C_j = S_{\text{readable}}, \quad C_j \cap C_k = \emptyset \text{ for } j \neq k. \quad (2)$$

This ownership rule prevents the same boundary material from becoming duplicate findings merely because it appeared in two windows. It is a structural invariant, not a completeness claim: an unreadable figure, a broken table order, or a reader’s missed clause still requires separate detection. A region inventory should label what was parsed, unsupported, or reopened from the original page.

The useful window size is not simply the largest context a model will accept. Small windows reduce per-call cost but can remove distant qualifiers; large windows can crowd the relevant passage and cause repeated work. A source-aware method should vary core size, halo width, and bridge review with source topology and task. It should preserve exact source IDs and short supporting spans for consequential statements. A writer need not receive every full source if a shared plan and local evidence packet carry the needed material, but a reviewer diagnosing an omission should be able to reopen the uncompressed source.

5 Methods and execution semantics

A method is a versioned proposal for how to do the job. It names its applicability conditions and exclusions, expected artifacts, worker roles, data passed to them, dependencies, resource limits, and review policy. It must be possible to reject the method for a task whose source modality or acceptance criterion it cannot handle. Current Lamina method files declare a bounded JSON graph and run through an operator-configured adapter. They do not select themselves, revise themselves, or inherit every source check from the guide builder.

Let V be the jobs of a run. Distinguish at least four relationships:

- A *hard-result edge* $u \rightarrow v$ means v waits for and receives u ’s completed result.
- A *shared-plan relation* means two jobs use the same versioned map or editorial decision, without requiring one writer’s finished prose.
- An *evidence-validity relation* links a source region or semantic decision to work that must be reconsidered if it changes; it does not automatically mean every descendant must be regenerated.
- A *resource relation* assigns work to a model, parser, renderer, review, or other lane with finite capacity. It can delay work without adding semantic information.

One untyped ‘depends on’ list cannot express all four without over-serializing or under-invalidating work. The current generic method runtime implements hard-result edges and resource lanes. The source-aware production path uses parallel local reading, a shared route, parallel sections, and review, but its plan is not previously finished prose. This distinction matters for callbacks and duplicated explanations: some writers need the shared plan and exact previous evidence; others genuinely need a prior finished section. Both patterns should remain available and be compared on the same task.

A source-aware planner should present alternative decompositions when evidence is weak. A source-order route retains local sequence and can be cheap for a short, coherent file. A shared topic board can coordinate parallel sections across multiple overlapping files, but its global reconciliation call may dominate latency or erase a distinction. A dependency chain of finished sections can improve continuity and exact callbacks, but it serializes writing and expands later context. A cross-section editor over parallel drafts can preserve overlap while paying an additional review and repair cost. The planner’s choice should cite task conditions—source topology, output form, conflict density, context budget, and delivery deadline—rather than merely selecting the strategy with the most nodes. Keeping rejected

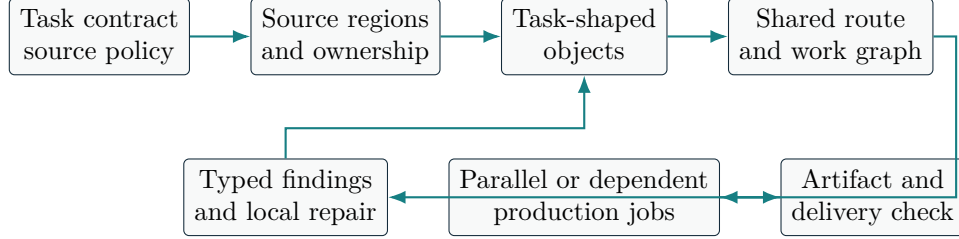


Figure 1: Intended source-aware production loop. The diagram separates semantic decisions from scheduling and delivery. Review may reopen a specific object or source decision; it need not restart every job. The fully connected loop is a design target, not a claim that all boxes are automated in the current release.

alternatives visible makes later repair intelligible: a failed source-order result may call for a different representation, not another prompt sentence.

6 Context assembly as an explicit decision

A worker’s context is selected, not inherited indiscriminately. Let I_v be the candidate items for job v and $P_v \subseteq I_v$ the protected items required by the task or a consequential claim. Each item i has measured size b_i in the adapter’s tokenizer. If the available input budget after instructions and output reserve is B_v , assembly chooses $X_v \subseteq I_v$ such that

$$P_v \subseteq X_v, \quad \sum_{i \in X_v} b_i \leq B_v. \quad (3)$$

A relevance score can rank optional items, but it is not a safety guarantee. If the protected set alone exceeds the budget, the job should split or ask for a scope decision; silent truncation violates the contract. Each packet should record the plan version, source and object IDs included, omitted candidate counts, and actual request size. The reviewer may need access to a broader inventory than a writer, precisely because a narrowed author packet cannot reveal what was lost before authoring.

The context-amplification ratio

$$A = \frac{\sum_{v \in V} \text{inputTokens}(v)}{\text{tokens}(\text{distinct accepted source text})} \quad (4)$$

is descriptive rather than an optimization target. Repeating a short protected rule across writers can prevent an error; duplicating an entire corpus in each local call can waste cost and attention. Report A by stage and alongside the largest request, source coverage, and accepted artifact quality. A low ratio achieved by starving workers of necessary context is not an improvement. The current method runner can select top-level task fields and direct dependency results; it does not yet implement the source-aware constrained selection in Eq. 3.

7 Concurrency and measured resource limits

For job service times t_v , total service work $W = \sum_v t_v$ and longest hard-dependency path D , the ideal lower bound on elapsed service time with p equivalent workers is

$$T \geq \max \left(D, \frac{W}{p} \right). \quad (5)$$

If lane r has capacity c_r and jobs using it require total lane service W_r , then a resource-aware lower bound is

$$T \geq \max \left(D, \frac{W}{p}, \max_r \frac{W_r}{c_r} \right). \quad (6)$$

These are classical work/span and capacity arguments, not new Lamina theorems; scheduling analyses such as work stealing assume particular task structures and costs [1]. Equation 6 assumes compatible service-time units and does not model simultaneous resource demands, provider throttling, request setup, queue wait, human review, retries, or delivery. It should locate possible bottlenecks, not predict a wall-clock completion time.

The design choice is where to put genuine dependency. Adding parallel writers cannot remove a long global reconciliation call from D . Requiring every writer to wait for completed earlier prose increases D , though it may lower duplication and repair. Resource contention can nullify a wider worker pool: a speech renderer and transcript checker sharing one memory-bound accelerator may be effectively serial even when model writers overlap. A finite batch graph does not necessarily meet the steady-state assumptions of a simple queueing formula. Instrument enqueue, start and end times, lane occupancy, retries and tail latency before using such a model.

A checked-in local benchmark exercises an older fixed graph of 24 simulated extraction jobs, two global steps, and six simulated author/review pairs. Its handlers only sleep and return tiny JSON objects. At worker limits 1, 4 and 8, cold wall times were 2.412, 0.865 and 0.595 seconds, respectively; warm cached runs took about 0.018–0.022 seconds with no handler calls. These measurements show scheduler overlap and cache reuse on one machine under artificial delays. They provide no evidence of model throughput, document quality, media delivery, or a general speed advantage. A separate integrated-path measurement is required after its implementation is frozen.

A second checked-in scheduler benchmark uses the same real SQLite/cache execution substrate with 64 sleep-only reading jobs, two serial global jobs and 16 sleep-only author/review chains, for 98 handler calls. Table 1 reports one local run per width, not a sampling distribution. The 32-worker setting reached 30 simultaneous handlers, not 32; the global stages and paired author/review dependencies limit overlap. Warm runs made zero handler calls but had non-monotone wall times as thread and cache bookkeeping remained. These measurements establish that configured width changes overlap under this fixture, not that a wider setting improves an inference workload.

8 Cache identity, review, and repair

A reusable job result must be keyed by its actual semantic request: source and policy versions, the job’s instructions and selected task fields, relevant dependency outputs, adapter identity, and any selected operator observation. Changing a label or an unrelated branch should not discard sound

Worker cap	Cold wall (s)	Peak handlers	Warm wall (s)	Warm calls
1	6.001	1	0.054	0
4	1.690	4	0.020	0
16	0.777	16	0.082	0
32	0.626	30	0.136	0

Table 1: Synthetic sleep workload on one local machine (Darwin arm64, Python 3.14.7, SQLite 3.53.4); one cold and one warm run per width. No model, human review, figure parsing or media delivery is in these times. Source: `benchmarks/results/production-widths.json`.

work; changing the source meaning or reviewer policy should not silently reuse an obsolete result. A content-addressed job store can make reuse explicit, but cache hits are not quality judgments. A stale reviewer verdict is especially dangerous when the artifact or source version it examined changes.

A review result should name the affected object and observed defect, not merely return ‘revise’. Examples include unreadable source region, missing answer object, false merge, unsupported claim, wrong condition, broken cross-section reference, candidate-answer leakage, and delivery failure. The finding needs an artifact location, source link, severity, reviewer identity, and exact artifact revision. Then the system can decide whether to reopen parsing, representation, selection, a writer, a renderer, or publication. Some findings need a human authority decision; a model should not resolve contradictory sources by confidence alone.

The dependency closure of a changed node is a candidate recheck set, not the exact repair bill. Let a repair receipt be

$$R(o) = (s_o, m_o, h_o, d_o), \quad (7)$$

where s_o is re-executed service seconds, m_o reported model or external charges, h_o human review minutes, and d_o replacement-delivery count for observation o . These values are deliberately not summed without an explicit conversion rule. A smaller invalidation set is useful only if the final artifact remains correct. Lamina currently caches stages and generic method nodes; the fixed guide can block export on review failure; and the production path can target a named section for revision, with a bounded section repair and re-review. Typed finding-to-upstream-cause routing and independent verification of final semantic quality remain limited; a generated repair is not automatically proof that its issue was fixed.

9 Experience and method reuse

Operational reuse and methodological learning are different. Operational reuse retrieves a node result whose semantic inputs are unchanged. Methodological learning changes a future decision: source policy, representation, window size, work graph, context packet, reviewer, or output form. Lamina’s method runtime stores run receipts and operator observations. An operator can explicitly attach a same-family observation ID to a later node, and that observation then enters the node’s context. This is a real downstream consumer of stored feedback. It is not automatic method selection or evidence that the observation improved the output.

A method revision should be a testable hypothesis: what failure it addresses, what it changes, what it preserves, and what new cost it may impose. Compare old and new versions on source collections

not used during tuning, with an explicit counterexample where the favored method could fail. Record losses as well as gains. Systems such as DSPy already study metric-driven optimization of language-model pipelines [4]; Lamina should not claim that optimizing prompts or saving reusable methods is novel. Its narrower research question is whether task-specific source and repair decisions improve accepted document production under realistic change.

10 Worked example: a disputed timeout

Consider a fictional engineering team preparing a short export-worker brief. Its owner-designated current specification says a worker receives a 90-second lease, renews it every 30 seconds while healthy, and stops producing output if renewal fails. It says the scheduler may reassign a job only after lease expiry. A historical runbook says operators once expected a 60-second timeout and records an incident in which a second worker began while the first appeared still to write. The runbook does not establish which timeout was configured during that incident or whether partial output was cleaned up. The team asks what to check before changing its retry timer.

A source-order summary could repeat both timeout numbers without explaining authority. A reader that aggressively merges them could produce one false rule. An evidence-led method instead gives each source an independent reader. The readers preserve atomic claims, short excerpts, source IDs and conditions. A reconciliation job applies the task owner’s authority policy, retains the old instruction as historical and the incident as an observation with unresolved cause, and produces a shared evidence board. A behavior writer and an operations writer use that same board in parallel. Their reviewer receives both drafts *and* the board, then flags a lost qualifier or unsupported recommendation. This is the repository’s portable six-node technical-brief example; the graph and context wiring have been structurally validated, but no model-written brief from that example has been independently accepted as an engineering answer.

Suppose the current specification changes its lease duration. The reader of that source and the reconciliation board are invalidated. Both writers may need fresh context, even if only one section visibly mentions the number. The historical reader can remain reusable if its selected task and source text are unchanged. A review finding about cleanup may instead target an information gap: the current specification is silent, and no amount of rewording should fabricate an answer. A good final brief should state the gap and recommend checking the actual cleanup contract. The repair trace should identify the new source version, reopened decisions, executed and reused jobs, reviewer result, and exact delivered brief. The present example demonstrates graph structure and local receipts, not that full conflict-to-delivery loop.

A separate deterministic production fixture exercises the actual source-aware engine with two original short source files and three planned sections. Its initial run reports one flagged section, one repair call, and no remaining flagged sections. On a named revision to section three, six of eight stage requests are cache hits; only section three’s write and review are misses, and the other two section bodies remain byte-identical. This tests request identity, targeted rerun and trace structure. The provider in this fixture returns canned JSON, so it cannot show that an LLM resolved the timeout conflict, wrote a useful brief or improved after review. The fixture’s millisecond timings are not production latency measurements.

Capability	Present mechanism	Status boundary
Source identity	Hashes, ordered text units, locators, adjacent extraction context	Working for supported text; not visual understanding
Fixed guide	Extract, reconcile, plan, author, review and export	Working application; one teaching route
Reusable method	User-declared JSON jobs, dependencies, lanes, cache and receipts	Working separately; semantic output rules external
Source-aware production	Bounded reading, shared route, section production and revision	Working local engine/API/CLI; semantic quality unproven
Experience	Operator-selected observations enter named future nodes	Explicit reuse; no automatic adaptation
Local delivery and recovery	HTML, Markdown, optional PDF, report, plan and project resume	Working locally; external publication separate
Typed upstream repair	Finding-to-cause routing across source, representation and output	Design target beyond section repair

Table 2: Status distinctions in the current checkout. Local end-to-end wiring and deterministic tests establish mechanics; they do not establish human-accepted model quality.

11 Current implementation and boundary

The repository has three related but unequal paths. The fixed guide builder ingests Markdown, text and extractable PDF text; stores source hashes, ordered units and locators; extracts with a short adjacent halo; globally reconciles and plans; authors and reviews lessons; and exports source-linked reading material. Validation checks certain source ownership, quotation, membership and output-shape properties. It cannot prove that parsing recovered visual meaning, that extraction found every important object, or that a citation entails an authored sentence. Its planned route and prerequisite evidence do not equal completed earlier prose.

The generic method runtime executes data-only JSON nodes with hard dependencies, task-field selection, worker and lane capacities, cache receipts, and operator-selected observations. Its generic response check requires a JSON object; declared expected shapes are hints, not fully enforced semantic contracts. The source-aware production engine reads selected teaching sources in bounded, owned windows with full neighboring units, asks a model for a natural section route and a representation per section, then writes and reviews sections under separate concurrency limits. One material review finding can trigger a section-local repair and re-review; unresolved findings remain visible. A named revision changes the selected section request and reuses unchanged cached work. The local project API persists state, marks unfinished work interrupted after restart, and can resume from retained plans and cache. The CLI and local Projects interface call the same engine. Export writes Markdown, HTML, a report, plan and optional PDF; assessment export separates candidate and examiner files. These mechanisms are implemented and structurally tested. They do not automatically infer prose-section dependencies after a revision, interpret arbitrary figures, perform a blind candidate solve, or prove audio or clinical quality. The generic method graph remains a separate execution path rather than replacing the production engine.

12 Evaluation and competitive position

The fair comparison is time and total cost to a *quality-matched accepted artifact*, including operator work. Freeze a new permissioned corpus and task contract before running. Include a difficult region such as a table or figure, overlap, one contradiction, and at least one known omission risk. Independent reviewers inventory important source objects, conditions and acceptable deliberate omissions from the rendered originals; preserve their disagreements. Hold any evaluation questions out of generation. Compare Lamina with a careful source-order baseline and, where feasible, fully configured external products under the same source and output rules. For an internal runtime comparison, keep the model/provider fixed; across products, report the combined system result rather than attributing a difference to orchestration alone.

Record first useful draft and accepted accessible artifact times, source-object misses and false merges, unsupported claims, editorial usability, model tokens, actual provider charges where available, human review minutes, retries, failed attempts and delivery checks. Distinguish estimated API-equivalent cost from observed incremental charge. Repeat paired tasks with rotated order and report median, tails, range and sample size. Then change one source, seed one local and one global finding, and ask a fresh operator to diagnose, repair and deliver again. Audit whether the smaller repair set was actually safe. A fresh-agent transfer run tests whether the saved method is usable without the original author. Structural tests, synthetic sleeps and a single polished example cannot establish semantic superiority.

Other products already provide capabilities adjacent to this design. Manus documents wide research and project context that evolves across tasks [8, 9]. NotebookLM provides source-based audio generation [3]. LangGraph and Google’s ADK support graph workflows; ADK’s newer documentation explicitly includes flexible graph and dynamic workflows [2, 6]. Product documentation does not disclose every internal mechanism, so Lamina should neither imply those systems lack graphs, memory or parallel research nor promise broad superiority. The plausible differentiated test is whether an operator can explain and revise a particular source-to-artifact decision with less accepted-work loss and less total effort. That remains a hypothesis.

13 Conclusion

A model is useful for reading, deciding, and writing; code is useful for exact identity, ownership, limits, scheduling, persistence and receipts. Neither can impersonate the other. Lamina’s engineering direction is to make the decisions between source and delivered artifact explicit enough to coordinate workers, preserve meaningful alternatives, and repair the right work after change. The guide, method and source-aware production paths now establish different pieces of this substrate, with local project delivery and recovery attached to production. Visual fidelity, output-specific representations beyond text sections, upstream finding repair, real delivery verification and comparative semantic quality remain open. The next milestone is a new-source demonstration with an actual correction and a measured accepted artifact, not a larger graph or a stronger claim.

References

- [1] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999. doi: 10.1145/324133.324234. URL <https://doi.org/10.1145/324133.324234>.
- [2] Google. Agent development kit: Workflows, 2026. URL <https://github.com/google/adk-docs/blob/main/docs/workflows/index.md>. Official ADK documentation, accessed 21 September 2026.
- [3] Google. Generate an audio overview in notebooklm, 2026. URL <https://support.google.com/gemininotebook/answer/16212820?hl=en>. Official product help, accessed 21 September 2026.
- [4] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html.
- [5] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 24370–24391, 2024. URL <https://proceedings.mlr.press/v235/kim24y.html>.
- [6] LangChain. Langgraph graph api, persistence, and interrupts, 2026. URL <https://docs.langchain.com/oss/python/langgraph/graph-api>. Official documentation, accessed 21 September 2026.
- [7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL <https://aclanthology.org/2024.tacl-1.9/>.
- [8] Manus. What is wide research?, 2026. URL <https://help.manus.im/en/articles/11960169-what-is-wide-research>. Official product documentation, accessed 21 September 2026.
- [9] Manus. Projects that learn from every task, 2026. URL <https://manus.im/blog/manus-projects-self-updating>. Official product description, accessed 21 September 2026.